COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithm | **SECTION: CPE21S3** | |
| **DATE PERFORMED:** | 9/17/26 | **DATE SUBMITTED:** | 9/17/26 |
| **NAME:** | Vince Andrew Urbano | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 9.1 Trees

### 1. PROCEDURE OUTPUT

ILO A:

Code

```
#ifndef GENERAL_TREE_H
#define GENERAL_TREE_H

#include <iostream>

template <typename T>
class CustomVector {
private:
    T* data;
    int capacity;
    int size;

    void resize(int new_capacity) {
        T* new_data = new T[new_capacity];
        for (int i = 0; i < size; ++i) {
            new_data[i] = data[i];
        }
        delete[] data;
        data = new_data;
        capacity = new_capacity;
    }

public:
    CustomVector() : data(nullptr), capacity(0), size(0) {}

    ~CustomVector() {
        delete[] data;
    }
```

```
    void push(const T& value) {
        if (size == capacity) {
            int new_capacity = (capacity == 0) ? 1 : capacity * 2;
            resize(new_capacity);
        }
        data[size++] = value;
    }

    T& operator[](int index) {
        return data[index];
    }

    const T& operator[](int index) const {
        return data[index];
    }

    int getSize() const {
        return size;
    }

    bool empty() const {
        return size == 0;
    }
};

template <typename T>
struct GeneralTreeNode {
    T data;
    CustomVector<GeneralTreeNode<T>*> children;

    GeneralTreeNode(T val) : data(val) {}
};
```

```
template <typename T>
class GeneralTree {
private:
    GeneralTreeNode<T>* root;

    void clear(GeneralTreeNode<T>* node) {
        if (!node) return;
        for (int i = 0; i < node->children.getSize(); ++i) {
            clear(node->children[i]);
        }
        delete node;
    }
```

```
public:
    GeneralTree() : root(nullptr) {}

    GeneralTree(T rootValue) {
        root = new GeneralTreeNode<T>(rootValue);
    }

    ~GeneralTree() {
        clear(root);
    }

    GeneralTreeNode<T>* getRoot() const {
        return root;
    }

    void setRoot(GeneralTreeNode<T>* node) {
        root = node;
    }

    GeneralTreeNode<T>* createNode(T val) {
        return new GeneralTreeNode<T>(val);
    }
```

```
137     void addChild(GeneralTreeNode<T>* parent, GeneralTreeNode<T>* child) {
138         if (parent && child) {
139             parent->children.push(child);
140         }
141     }
```

| | |
|---|---|
| Output | |

```cpp
#include <iostream>
#include "ILOA.h"

int main() {
    GeneralTree<char> tree( rootValue: 'A');
    GeneralTreeNode<char>* root = tree.getRoot();

    GeneralTreeNode<char>* b = tree.createNode( val: 'B');
    GeneralTreeNode<char>* c = tree.createNode( val: 'C');
    GeneralTreeNode<char>* d = tree.createNode( val: 'D');
    GeneralTreeNode<char>* e = tree.createNode( val: 'E');
    GeneralTreeNode<char>* f = tree.createNode( val: 'F');
    GeneralTreeNode<char>* g = tree.createNode( val: 'G');

    tree.addChild( parent: root,  child: b);
    tree.addChild( parent: root,  child: c);
    tree.addChild( parent: root,  child: d);
    tree.addChild( parent: root,  child: e);
    tree.addChild( parent: root,  child: f);
    tree.addChild( parent: root,  child: g);

    GeneralTreeNode<char>* h = tree.createNode( val: 'H');
    tree.addChild( parent: d,  child: h);

    GeneralTreeNode<char>* i = tree.createNode( val: 'I');
    GeneralTreeNode<char>* j = tree.createNode( val: 'J');
    tree.addChild( parent: e,  child: i);
    tree.addChild( parent: e,  child: j);
```

| | |
|---|---|
| | ```<br>GeneralTreeNode<char>* k = tree.createNode( val: 'K');<br>GeneralTreeNode<char>* l = tree.createNode( val: 'L');<br>GeneralTreeNode<char>* m = tree.createNode( val: 'M');<br>tree.addChild( parent: f,  child: k);<br>tree.addChild( parent: f,  child: l);<br>tree.addChild( parent: f,  child: m);<br><br>GeneralTreeNode<char>* n = tree.createNode( val: 'N');<br>tree.addChild( parent: g,  child: n);<br><br>GeneralTreeNode<char>* p = tree.createNode( val: 'P');<br>GeneralTreeNode<char>* q = tree.createNode( val: 'Q');<br>tree.addChild( parent: j,  child: p);<br>tree.addChild( parent: j,  child: q);<br>``` |
| Observation | In this code, I have noticed that it creates more than two children from the root which is very different from the binary search tree. |

Task2:

| Node | Height | Depth |
|---|---|---|
| A | 3 | 0 |
| B | 0 | 1 |
| c | 0 | 1 |
| D | 1 | 1 |
| E | 2 | 1 |
| F | 1 | 1 |
| G | 1 | 1 |
| H | 0 | 2 |
| I | 0 | 2 |
| J | 1 | 2 |
| K | 0 | 2 |
| L | 0 | 2 |
| M | 0 | 2 |
| N | 0 | 2 |
| P | 0 | 3 |
| Q | 0 | 3 |

ILOB:

| | |
|---|---|
| Pre-Order | A B C D H E I J P Q F K L M G N |
| In-Order | B A C H D I E P J Q K F L M N G |
| Post-Order | B C H D I P Q J E K L M F N G A |

Pre-Order Function:

```cpp
template <typename T>
void preOrder(GeneralTreeNode<T>* node) {
    if (!node) return;
    std::cout << node->data << " ";
    for (int i = 0; i < node->children.getSize(); ++i) {
        preOrder(node->children[i]);
    }
}
```

```cpp
std::cout << "PreOrder: " << std::endl;
preOrder(root);
std::cout << "\n";
```

```
PreOrder:
A B C D H E I J P Q F K L M G N
```

In-Order:

```cpp
template <typename T>
void inOrder(GeneralTreeNode<T>* node) {
    if (!node) return;

    int totalChildren = node->children.getSize();

    if (totalChildren == 0) {
        std::cout << node->data << " ";
        return;
    }

    inOrder(node->children[0]);

    std::cout << node->data << " ";

    for (int i = 1; i < totalChildren; ++i) {
        inOrder(node->children[i]);
    }
}
```

```cpp
 std::cout << "In Order: " << std::endl;

 inOrder(root);
 std::cout << "\n";
```

```
In Order:
B A C H D I E P J Q K F L M N G
```

Post Order:

```cpp
template <typename T>
void postOrder(GeneralTreeNode<T>* node) {
    if (!node) return;
    for (int i = 0; i < node->children.getSize(); ++i) {
        postOrder(node->children[i]);
    }
    std::cout << node->data << " ";
}
```

```cpp
std::cout << "Post Order: " << std::endl;
postOrder(root);
std::cout << "\n";
```

```
Post Order:
B C H D I P Q J E K L M F N G A
```

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.
Code:

```cpp
template<typename T>
class BinarySearchTree{
    private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    public:
    BinarySearchTree(T value);
    //insertion:
    void insert(T value);

    //traversal:
    void preOrder();
    void inOrder();
    void postOrder();

};
```

```cpp
template <typename T>
void BinarySearchTree<T>::insert(T value){
    //go to left if  value < key
    if (value < key){
        if (left == nullptr){
            left = new BinarySearchTree<T>(value);
        }

        else{
        left->insert(value);
        }
    }
    //go to right if value > key
    else if (value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right->insert(value);
        }
    }
}
```

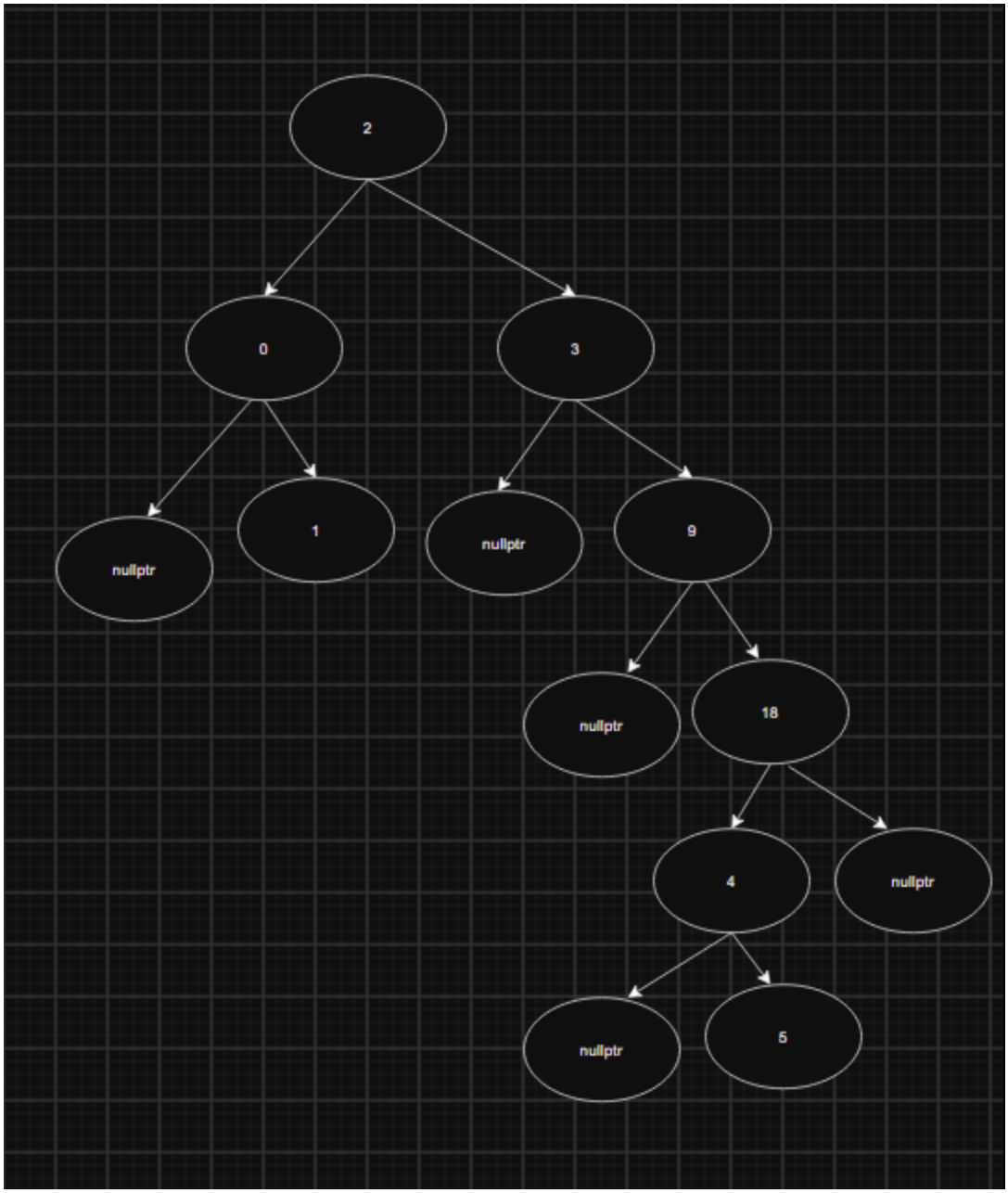
```
#include <iostream>
#include "BST.h"


int main(){

    BinarySearchTree<int>* tree = new BinarySearchTree(2);

    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.




Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```cpp
template <typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout << key << " ";

    if (right != nullptr){
        right->inOrder();
    }
}
```

```cpp
std::cout <<"In Order Output: ";
tree->inOrder();
std::cout<<std::endl;
```

```
In Order Output: 0 1 2 3 4 5 9 18
```

Pre-order Traversal

```cpp
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";

    if(left != nullptr){
        left->preOrder();
    }
    if (right != nullptr){
        right->preOrder();
    }
}
```

```cpp
std::cout <<"Pre Order Output: ";
tree->preOrder();
std::cout<<std::endl;
```

```
Pre Order Output: 2 0 1 3 9 4 5 18
```

Post-Order Traversal

```
76  template <typename T>
77  void BinarySearchTree<T>::postOrder(){
78      if(left != nullptr){
79          left->postOrder();
80      }
81
82      if (right != nullptr){
83          right->postOrder();
84      }
85      std::cout << key << " ";
86
87  }
```

```
std::cout <<"Post-Order Output: ";
tree->postOrder();
std::cout<<std::endl;
```

```
Post-Order Output: 1 0 5 4 18 9 3 2
```

Conclusion:
During this activity, I have learned how pre-order, in-order and post-order differ from each other. The pre-order traversal starts with the from the root and will move to the left child to the right. In-order works aggressively by starting from the very left to the root and to the right child of the root. Lastly, post order works by checking the child first before the root, from left to right to root.

**3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT**

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.